

StreamFlow

Практическое руководство: от Docker-образа до работающего кластера

Kubernetes для джуниоров

Предварительные требования

Убедитесь что на вашей машине установлено следующее:

Проверка: Выполните все команды ниже перед началом. Если что-то не работает — скажите преподавателю.

Инструмент	Команда проверки	Ожидаемый результат
Docker	<code>docker --version</code>	Docker version 24.x или выше
minikube	<code>minikube version</code>	minikube version: v1.x.x
kubectl	<code>kubectl version --client</code>	Client Version: v1.x.x

Часть 1. Сборка Docker-образа

Весь StreamFlow упакован в один образ. В зависимости от переменной окружения `SERVICE_NAME` контейнер запускается в нужной роли. Это удобно для обучения — один образ, разное поведение.

Шаг 1. Склонируйте репозиторий

```
git clone <url-репозитория>
cd streamflow
```

Убедитесь что в папке есть три файла:

- `main.go` — исходный код всех сервисов
- `Dockerfile` — инструкция сборки образа
- `streamflow.yaml` — манифест кластера

Шаг 2. Изучите Dockerfile

Откройте Dockerfile и найдите ответы на вопросы:

- Какой базовый образ используется?
- Как компилируется Go-приложение?
- Какой порт слушает приложение внутри контейнера?

Подсказка: Go-приложение компилируется командой `go build`. Итоговый бинарник копируется в чистый образ — это называется `multi-stage build`. Итоговый образ весит намного меньше.

Шаг 3. Подключитесь к Docker внутри minikube

minikube запускает собственный Docker daemon. Чтобы собранный образ был доступен Kubernetes, нужно собрать его именно там:

```
# Перенаправляем команды docker на daemon внутри minikube
eval $(minikube docker-env)

# Проверка — должны появиться системные образы minikube
docker images
```

Важно: Эта команда действует только в текущей вкладке терминала. Если откроете новую — нужно выполнить снова.

Шаг 4. Соберите образ

```
docker build -t streamflow-mock:v1 .
```

Флаг `-t` задаёт имя и тег образа. Имя `streamflow-mock:v1` используется в манифесте `streamflow.yaml` — они должны совпадать.

Проверьте что образ появился:

```
docker images | grep streamflow
```

Ожидаемый вывод: `streamflow-mock v1 <id> <время> <размер>`

Часть 2. Запуск кластера minikube

Шаг 5. Запустите minikube

```
minikube start
```

Первый запуск занимает 2-5 минут. minikube скачает образ виртуальной машины и поднимет однонодовый кластер. В конце должно появиться:

Ожидаемый вывод: Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

Шаг 6. Проверьте что кластер живой

```
# Посмотреть ноды кластера
kubectl get nodes

# Посмотреть системные поды
kubectl get pods -n kube-system
```

У ноды должен быть статус Ready. Если статус NotReady — подождите ещё 30 секунд и повторите.

Шаг 7. Включите Ingress (понадобится в Части 4)

```
minikube addons enable ingress

# Дождитесь пока ingress-nginx-controller перейдёт в Running
kubectl get pods -n ingress-nginx --watch
```

Нажмите Ctrl+C когда увидите статус Running.

Часть 3. Модуль 1 — Pod, Deployment, Liveness Probe

Шаг 8. Изучите манифест перед применением

Откройте streamflow.yaml и найдите:

Что искать	Вопрос
Секция livenessProbe	Сколько секунд ждёт Kubernetes перед первой

	проверкой?
Параметр <code>replicas</code>	Сколько копий <code>catalog-service</code> будет запущено?
Поле <code>imagePullPolicy: Never</code>	Почему стоит <code>Never</code> , а не <code>Always</code> ?
Поле <code>selector.matchLabels</code>	Как <code>Deployment</code> знает какими подами управлять?

Шаг 9. Примените манифест

```
kubectl apply -f streamflow.yaml
```

Kubernetes прочитает файл и создаст все объекты описанные в нём: `ConfigMap`, `Deployments`, `Services`, `Ingress`.

Что делает `apply`: `kubectl apply` — декларативная команда. Kubernetes сравнивает желаемое состояние из файла с текущим и применяет только разницу. Можно выполнять многократно — повторное применение безопасно.

Шаг 10. Наблюдайте за запуском подов

```
# Режим watch — обновляется каждые 2 секунды
kubectl get pods --watch
```

Нажмите `Ctrl+C` когда все поды перейдут в статус `Running`. Ожидаемый результат:

NAME	READY	STATUS	RESTARTS	AGE
<code>api-deploy-xxxx</code>	1/1	Running	0	30s
<code>catalog-deploy-xxxx</code>	1/1	Running	0	30s
<code>catalog-deploy-xxxx</code>	1/1	Running	0	30s
<code>catalog-deploy-xxxx</code>	1/1	Running	0	30s
<code>transcoder-deploy-xxxx</code>	1/1	Running	0	30s

Состояния подов: `Pending` — ждёт ресурсов. `ContainerCreating` — скачивает образ и создаёт контейнер. `Running` — работает. `CrashLoopBackOff` — падает в цикле, смотрите логи.

Шаг 11. Демо: Pod и Deployment

Выполните команды по очереди и наблюдайте за результатом:

11.1 Посмотреть детали пода

```
# Замените <pod-name> на реальное имя из kubectl get pods
```

```
kubectl describe pod <pod-name>
```

Найдите секцию Events в самом низу вывода. Там вся история пода: когда создался, когда liveness probe проверяла, когда что-то пошло не так.

11.2 Посмотреть логи

```
kubectl logs <pod-name>

# Следить за логами в реальном времени
kubectl logs <pod-name> --follow
```

11.3 Удалить под — наблюдать за автовосстановлением

```
# Запустите в соседней вкладке перед удалением
kubectl get pods --watch

# Удалите один из catalog-подов
kubectl delete pod <catalog-pod-name>
```

Наблюдайте: под исчезнет и через несколько секунд появится новый с другим суффиксом. Deployment заметил что реплик стало меньше нужного и создал замену.

11.4 Симулировать сбой сервиса

```
# Узнать IP пода
kubectl get pod <transcoder-pod-name> -o wide

# Войти внутрь пода и вызвать /fail
kubectl exec -it <transcoder-pod-name> -- wget -q -O-
http://localhost:8080/fail
```

Под упадёт (log.Fatal внутри приложения). Наблюдайте в соседней вкладке как Deployment пересоздаёт его. Счётчик RESTARTS увеличится на 1.

Почему RESTARTS растёт: Каждый раз когда контейнер завершается с ненулевым кодом — Kubernetes перезапускает его. RESTARTS показывает сколько раз это случилось. Большое значение — сигнал что с подом что-то не так.

Часть 4. Модуль 2 — Service и DNS

Шаг 12. Проверьте Service

```
kubectl get services
```

Вы увидите catalog-service и api-service с типом ClusterIP. Это значит они доступны только внутри кластера — снаружи не видны.

Шаг 13. Убедитесь что DNS работает

Зайдите внутрь api-пода и обратитесь к catalog-service по DNS-имени:

```
kubectl exec -it <api-pod-name> -- sh

# Внутри контейнера:
wget -q -O- http://catalog-service/list
exit
```

Ожидаемый ответ — JSON со списком фильмов. Обратите внимание на поле node — это имя пода catalog, который ответил на запрос. При нескольких запросах подряд значение будет меняться — это балансировка нагрузки.

Как работает DNS в Kubernetes: CoreDNS автоматически создаёт запись для каждого Service. Имя catalog-service резолвится в виртуальный IP сервиса, который балансирует запросы между всеми подами с лейблом app: catalog.

Шаг 14. Проверьте Ingress

```
# В отдельной вкладке — держите открытым
minikube tunnel
```

```
# В основной вкладке
curl http://localhost/
```

Вы должны увидеть HTML-страницу StreamFlow Gateway с именами подов. Это запрос прошёл путь: curl → Ingress → api-service → api-pod → catalog-service → catalog-pod.

Попробуйте обратиться к catalog напрямую — не получится:

```
curl http://localhost/list
```

Почему: Ingress настроен только на путь /. Прямого доступа к catalog-service снаружи нет — он ClusterIP.

Часть 5. Модуль 3 — ConfigMap

Шаг 15. Проверьте ConfigMap

```
kubectl get configmap stream-config -o yaml
```

Вы увидите ключ WELCOME_MSG со значением "StreamFlow v1.0". Это данные хранящиеся в кластере отдельно от образа.

Шаг 16. Посмотрите переменную окружения внутри пода

```
kubectl exec -it <api-pod-name> -- env | grep WELCOME
```

Значение придёт из ConfigMap. Под не знает и не должен знать откуда взялась переменная — просто видит её в окружении.

Шаг 17. Измените ConfigMap и обновите поды

```
# Открыть редактор в браузере  
kubectl edit configmap stream-config
```

Найдите строку WELCOME_MSG и измените значение, например на "StreamFlow v2.0". Сохраните и закройте редактор.

ConfigMap обновится — но поды уже запущены со старыми значениями. Перезапустите их:

```
kubectl rollout restart deployment/api-deploy  
  
# Дождитесь завершения  
kubectl rollout status deployment/api-deploy
```

Проверьте что новое значение подхватилось:

```
kubectl exec -it <новый-api-pod-name> -- env | grep WELCOME
```

Вывод: Образ контейнера не менялся. Изменился только ConfigMap. Это и есть разделение кода и конфигурации.

Часть 6. Модуль 5 — Масштабирование

Шаг 18. Масштабируйте transcoder вручную

```
# Увеличьте до 5 реплик
kubectl scale deployment/transcoder-deploy --replicas=5

# Наблюдайте как поды появляются
kubectl get pods --watch
```

Должно появиться 5 подов transcoder. Нажмите Ctrl+C когда все перейдут в Running.

Шаг 19. Посмотрите распределение по нодам

```
kubectl get pods -o wide
```

Колонка NODE показывает на какой ноде запущен каждый под. В minikube нода одна, но в реальном кластере Kubernetes сам распределяет поды между несколькими нодами.

Шаг 20. Уменьшите обратно

```
kubectl scale deployment/transcoder-deploy --replicas=1
```

Kubernetes выберет какие поды остановить. Оставшийся под продолжит работу без прерываний.

Часть 7. Модуль 6 — RollingUpdate и откат

Шаг 21. Задеплойте "новую версию"

Сымитируем выкатку новой версии, изменив тег образа на несуществующий:

```
kubectl set image deployment/catalog-deploy server=streamflow-mock:v999
```

Шаг 22. Наблюдайте за RollingUpdate

```
kubectl rollout status deployment/catalog-deploy

# В соседней вкладке
kubectl get pods --watch
```

Kubernetes попытается поднять новые поды с тегом v999. Так как образа не существует — они застрянут в ImagePullBackOff. Но старые поды с v1 останутся живыми и продолжат обслуживать трафик.

Ключевой момент: Kubernetes не убивает старые поды пока новые не пройдут liveness probe. Если деплой сломан — сервис продолжает работать на старой версии.

Шаг 23. Откатитесь назад

```
kubectl rollout undo deployment/catalog-deploy

# Убедитесь что откатились на v1
kubectl rollout status deployment/catalog-deploy
kubectl get pods
```

Через несколько секунд все поды вернуться к рабочей версии. Проверьте что API снова работает:

```
curl http://localhost/
```

Шаг 24. Посмотрите историю деплоев

```
kubectl rollout history deployment/catalog-deploy
```

Kubernetes хранит историю ревизий. Можно откатиться на конкретную:

```
kubectl rollout undo deployment/catalog-deploy --to-revision=1
```

Шпаргалка — команды которые решают 90% проблем

Команда	Что делает
<code>kubectl get pods</code>	Статус всех подов. Открывать первым при любой проблеме.
<code>kubectl get pods --watch</code>	Обновляется в реальном времени. Ctrl+C для выхода.
<code>kubectl describe pod <name></code>	Подробности пода + секция Events с историей.
<code>kubectl logs <name></code>	Логи контейнера. Добавьте --follow для стриминга.
<code>kubectl logs <name> --previous</code>	Логи предыдущего контейнера (если под рестартовал).
<code>kubectl exec -it <name> -- sh</code>	Войти внутрь контейнера в интерактивный шелл.
<code>kubectl apply -f file.yaml</code>	Применить манифест. Идемпотентно.
<code>kubectl delete -f file.yaml</code>	Удалить все объекты из манифеста.
<code>kubectl scale deployment/<name> -- replicas=N</code>	Изменить число реплик.
<code>kubectl rollout undo deployment/<name></code>	Откатить деплой на предыдущую версию.
<code>kubectl rollout history deployment/<name></code>	История деплоев и ревизий.
<code>kubectl get events --sort-by='.lastTimestamp'</code>	Все события кластера, свежие внизу.

Типичные проблемы

Статус / Ошибка	Причина	Решение
ImagePullBackOff	Образ не найден в реестре или опечатка в теге	<code>kubectl describe pod</code> — смотрите Events. Проверьте имя образа и что <code>eval \$(minikube docker-env)</code> был выполнен.
CrashLoopBackOff	Контейнер падает сразу после запуска	<code>kubectl logs <pod> --previous</code> — смотрите последний вывод перед падением.
Pending (долго)	Нет ресурсов или нода NotReady	<code>kubectl describe pod</code> — секция Events покажет причину. Проверьте <code>minikube status</code> .

Catalog unreachable: 500	api-pod не может достучаться до catalog- service	Проверьте <code>kubectl get services</code> и что catalog поды в Running.
minikube tunnel зависает	Требует sudo или занят порт	Запустите в отдельном терминале. На Linux может потребовать пароль.

После занятия: остановите кластер командой `minikube stop`